

International Cybersecurity and Digital Forensics Academy (ICDFA)

SBT-DF203: Basic Networking Skills for Digital Forensics

Lab 8: DNS Spoofing Forensics — Practical Report

Student Name	Mary-Joy Adewole
Registration Number	2025/FWSD/11468
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Instructor	Aminu Idris, AMCPN
Practical Title	Lab 8 — DNS Spoofing Forensics
Delivery Block	3/3 — 19–25 September 2026
Environment	Kali Linux analyst VM + Kali Linux victim VM, VMware Workstation, host-only network (192.168.74.0/24)

1. Executive Summary and Authorization

This report documents an authorized, host-only DNS spoofing forensics exercise performed for ICDFA course SBT-DF203, Lab 8. The exercise reproduced a controlled ARP poisoning and DNS spoofing scenario against a dedicated victim VM, using a reserved, non-existent training domain (portal.icdfa.test) and a harmless warning page. No real domains, third-party systems, or credentials were targeted at any point. The exercise was performed under the authorization granted by the lab assignment itself, issued by the course instructor, Aminu Idris.

A pre-lab review of the instructor-supplied script scripts/dns_spoof.py found that its default configuration targeted real third-party domains (google.com, facebook.com, ubalt.com, ubalt.edu, boogle.com) rather than the reserved training domain. This was identified and corrected before any spoofing script was executed, in line with the lab's safety requirements. The correction is documented in Section 6 and Appendix A.

2. Lab Network Diagram

Two Kali Linux virtual machines were connected on a VMware Workstation host-only network (192.168.74.0/24):

- Analyst VM (kali-linux-2026.2) — eth0: 192.168.74.128/24 (host-only, victim-facing); eth1: NAT (internet access for package installs only, not used in the attack path).
- Victim VM (Kali-Client) — eth0: 192.168.74.129/24 (host-only).
- Gateway (virtual router, not a real host): 192.168.74.1.
- Training DNS resolver (dnsmasq) running on the analyst VM, bound to 192.168.74.128:53, answering only for portal.icdfa.test.
- Apache2 web server on the analyst VM, serving the harmless ICDFA training warning page at 192.168.74.128:80.

3. Baseline Evidence and Hashes

Before any spoofing activity, the environment was set up and a legitimate DNS/ARP/HTTP baseline was captured.

3.1 Environment setup

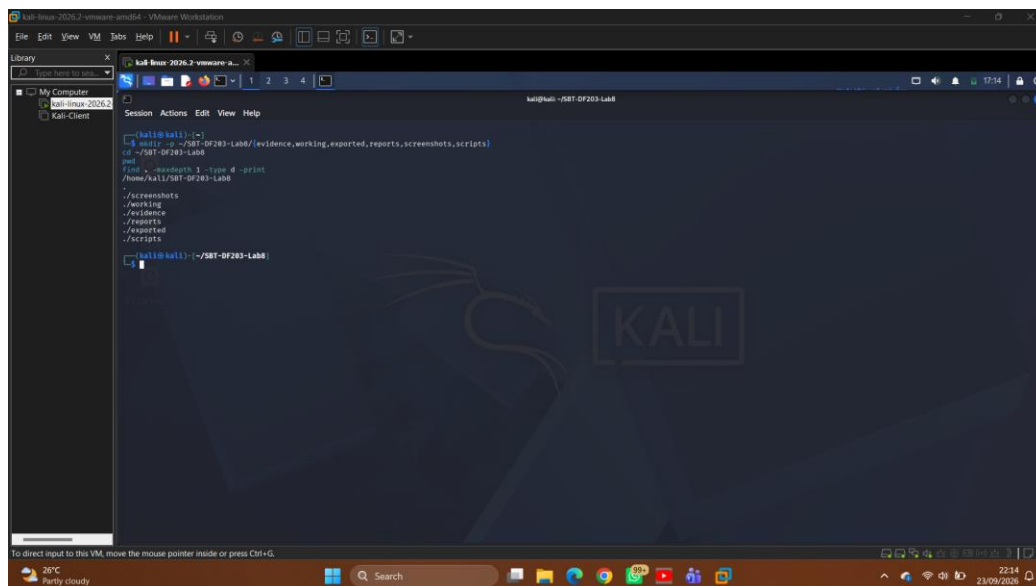


Figure 1: Lab folder structure created under ~/SBT-DF203-Lab8 on the analyst VM.

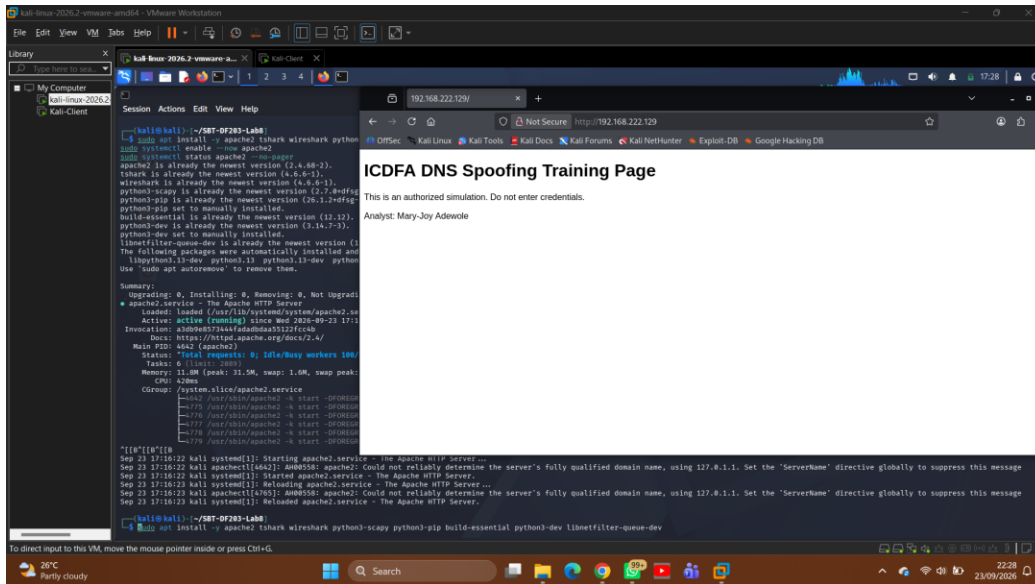


Figure 2: Required packages installed (apache2, tshark, wireshark, scapy, build tools) and apache2 service active.

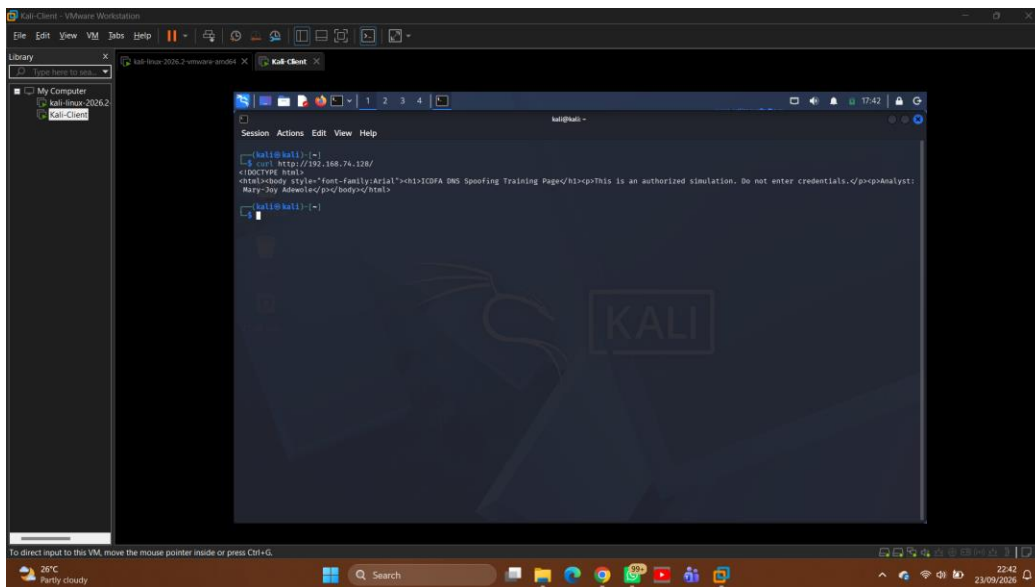


Figure 3: Harmless ICDFA training warning page rendered in-browser, confirming no credential fields and correct analyst attribution.

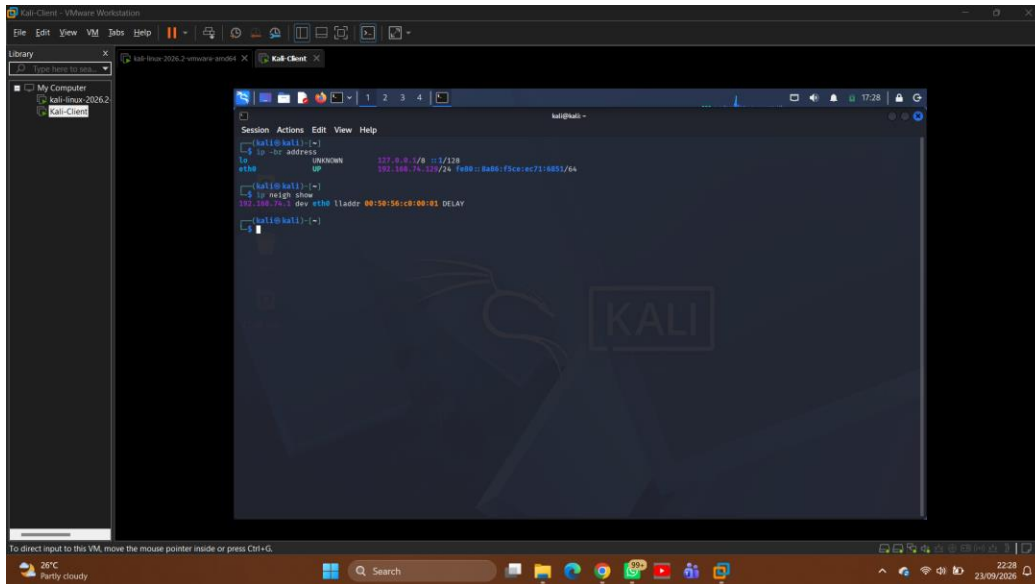


Figure 4: Initial interface, route, and ARP-table state recorded on the analyst VM prior to any lab activity.

3.2 Training DNS resolver (dnsmasq)

Because portal.icdfa.test is a reserved, non-existent domain, a dedicated training resolver (dnsmasq) was configured on the analyst VM, bound only to the host-only interface (eth0), to provide a legitimate baseline answer for the domain.

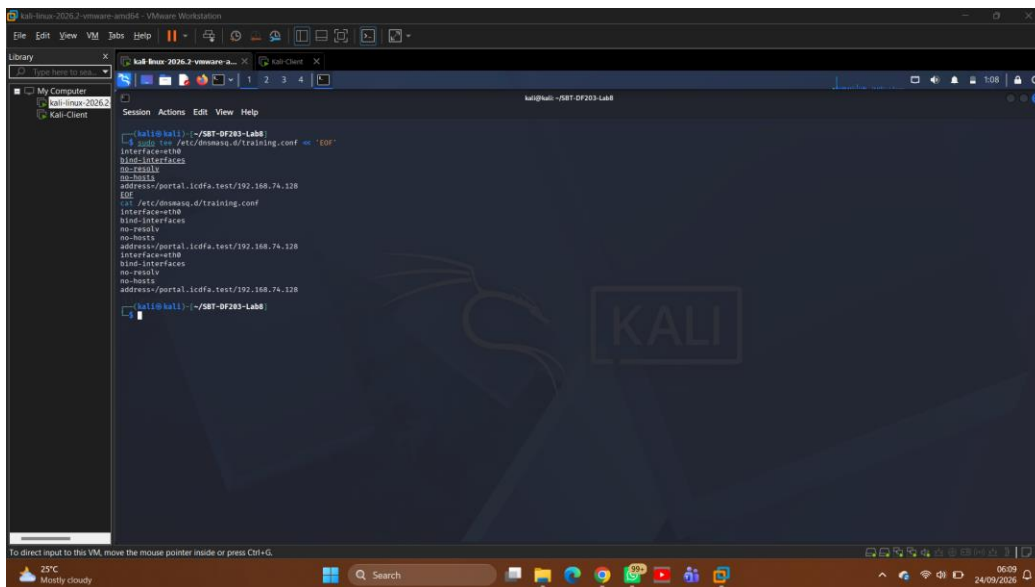


Figure 5: dnsmasq configuration — interface=eth0, bind-interfaces, address=/portal.icdfa.test/192.168.74.128.

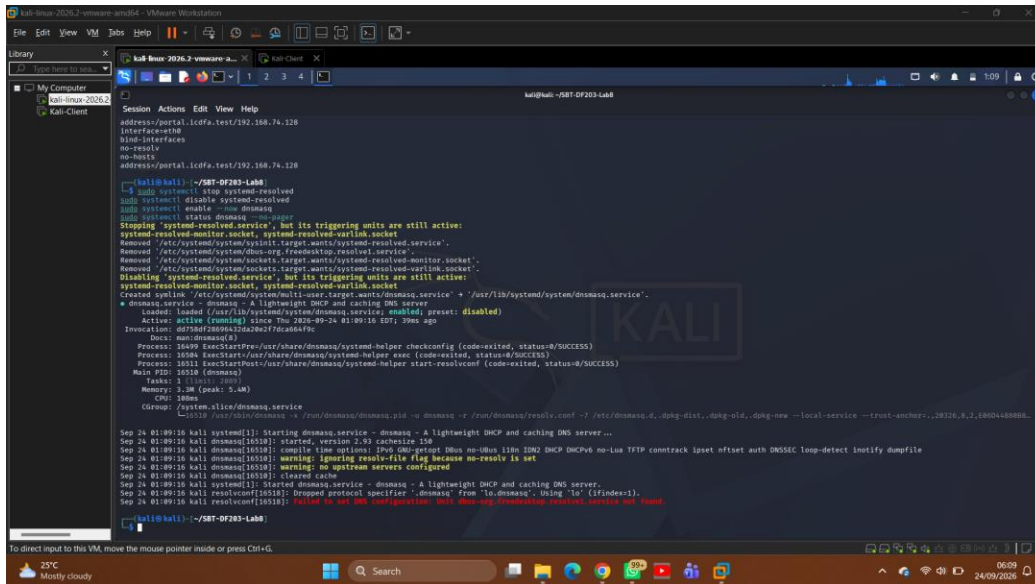


Figure 6: dnsmasq service active and running on the analyst VM.

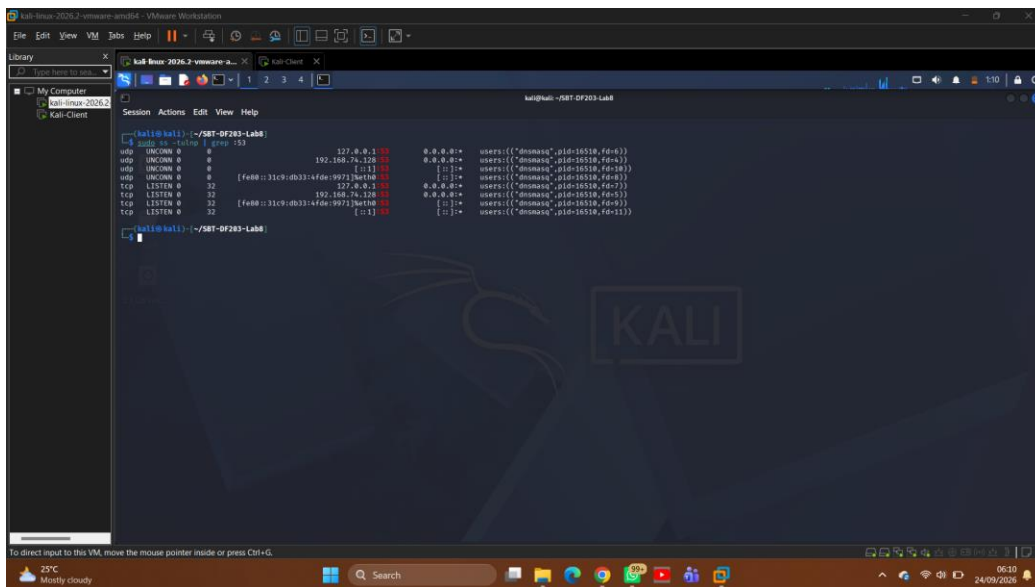


Figure 7: dnsmasq confirmed listening on 127.0.0.1:53 and 192.168.74.128:53 only.

3.3 Baseline DNS resolution and capture

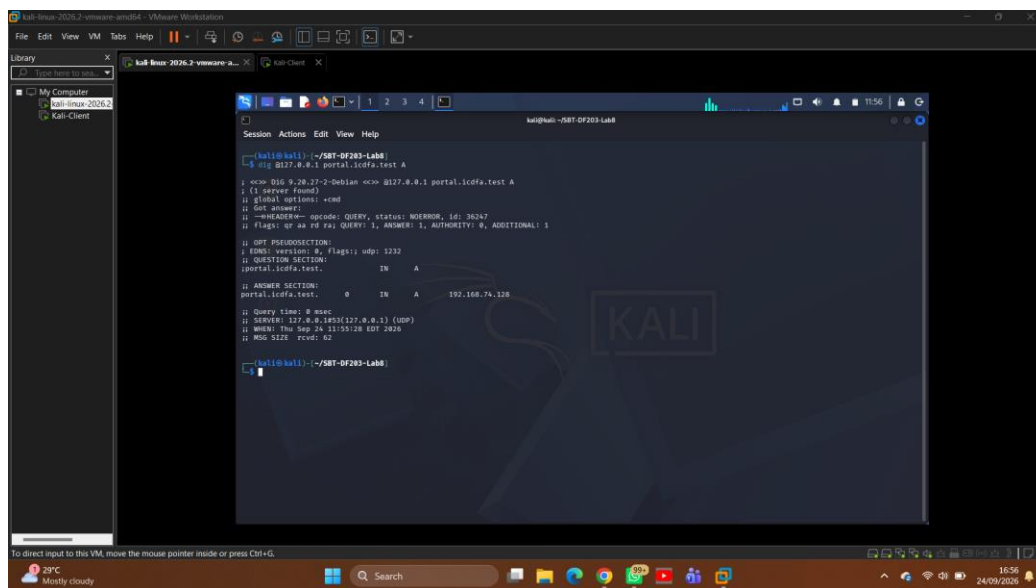


Figure 8: Victim VM successfully resolves `portal.icdfa.test` to `192.168.74.128` via the training resolver (NOERROR, `dig @192.168.74.128`).

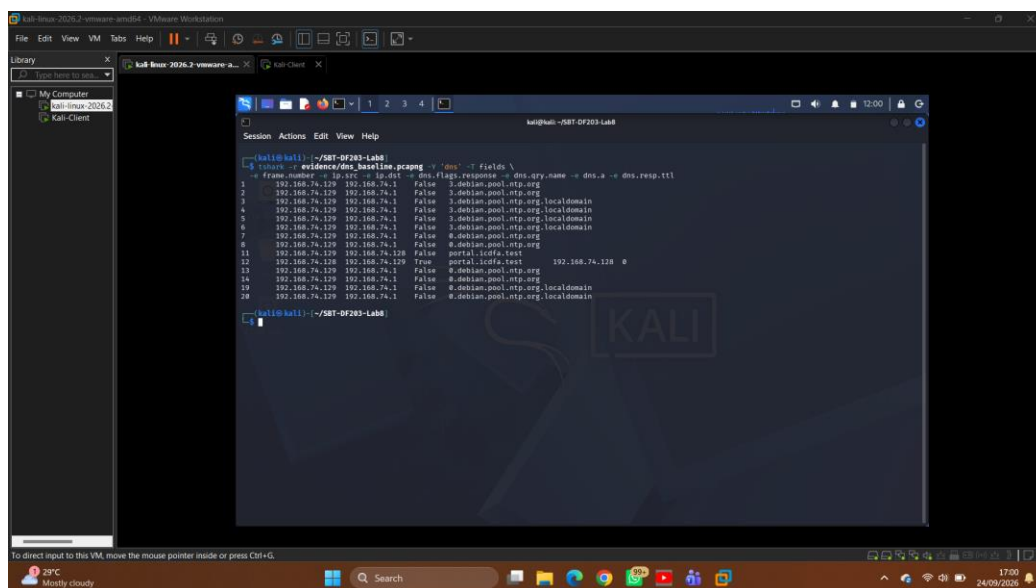


Figure 9: Baseline capture (`dns_baseline.pcapng`) completed and SHA-256 hashed.

Baseline evidence file: `evidence/dns_baseline.pcapng`

Baseline SHA-256: `da1214d76e1fd25e41ee3df759fd9017b281a3f5311c751012bad70c8d45f21f`

Baseline finding: responder IP `192.168.74.128`, responder confirmed as the analyst's own `dnsmasq` instance, answer `192.168.74.128`, TTL `0`, transaction ID matched between query and response (frames 11–12 of the baseline capture).

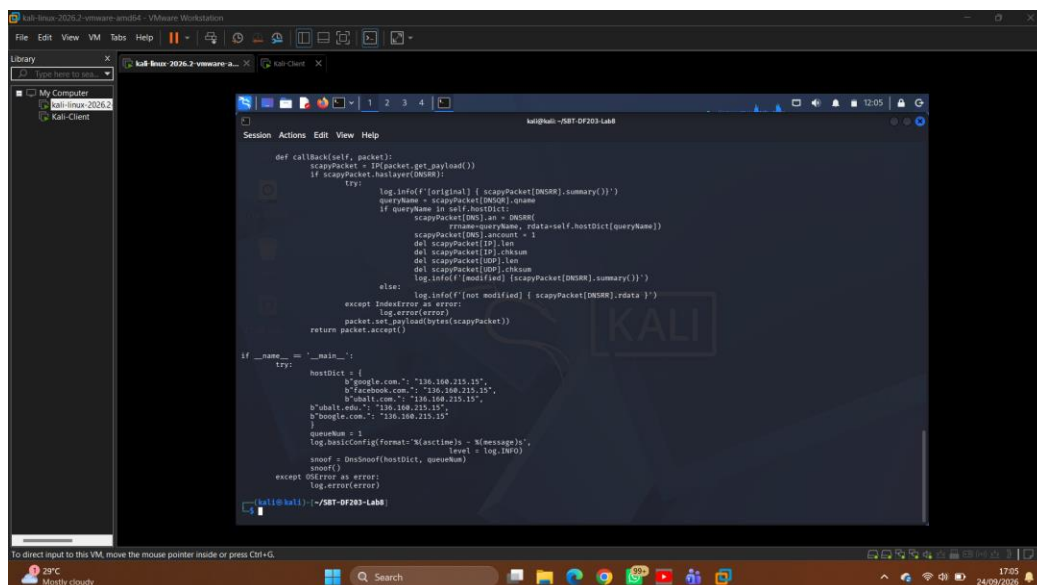


Figure 11: *dns_spoof.py* reviewed — default `hostDict` found to target real third-party domains (*google.com*, *facebook.com*, *ubalt.com*, *ubalt.edu*, *boogle.com*) at 136.160.215.15, in violation of the lab's safety requirement to target only the reserved training domain.

This was corrected before execution: the `hostDict` was replaced with a single mapping of `portal.icdfa.test` to the analyst VM's IP (192.168.74.128). A backup of the original script was kept (`scripts/dns_spoof_original_backup.py`) and a diff of the change was recorded as evidence.

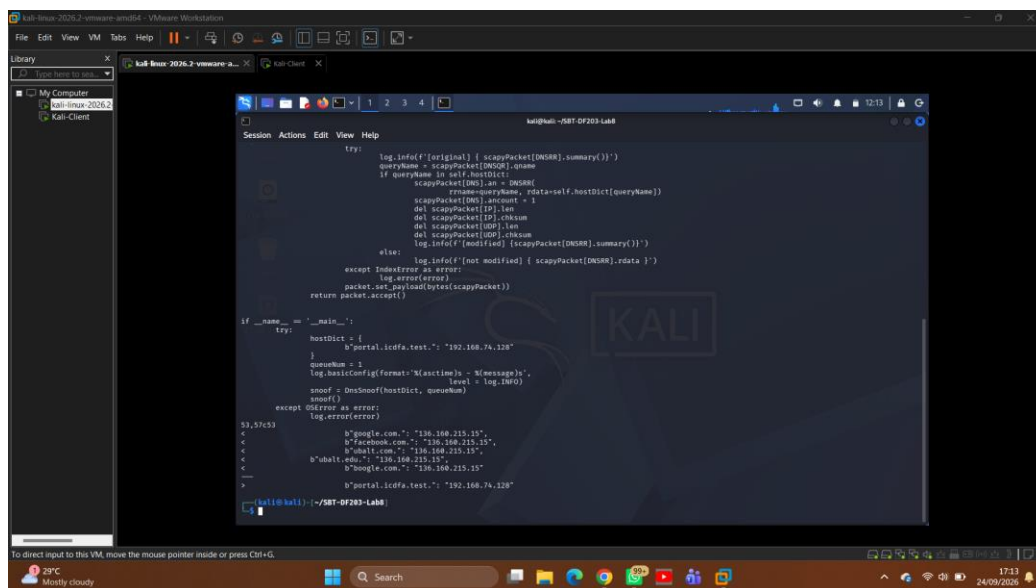


Figure 12: Diff confirming the five real-domain entries were removed and replaced with the single authorized training-domain mapping.

Supporting software: NetfilterQueue (Python binding for libnetfilter-queue) was installed to support `dns_spoof.py`'s packet-interception mechanism, which uses an iptables NFQUEUE rule to intercept and rewrite in-flight DNS responses.

5. Controlled Spoofing Event and DNS/ARP Timeline

The controlled simulation was run as three concurrent processes on the analyst VM, with the victim VM triggering DNS resolution and an HTTP request partway through:

```

Terminal 1 (capture):
sudo tshark -i eth0 -f 'host 192.168.74.129 and (arp or port 53 or tcp port 80)' \
-a duration:60 -w evidence/dns_spoof_controlled.pcapng

Terminal 2 (ARP poisoning):
sudo timeout 30 python3 scripts/arp.py 192.168.74.129 192.168.74.1

Terminal 3 (DNS interception/spoof, ~5s after Terminal 2):
sudo timeout 25 python3 scripts/dns_spoof.py

Victim (during the above): dig portal.icdfa.test A ; browse to http://portal.icdfa.test/

```

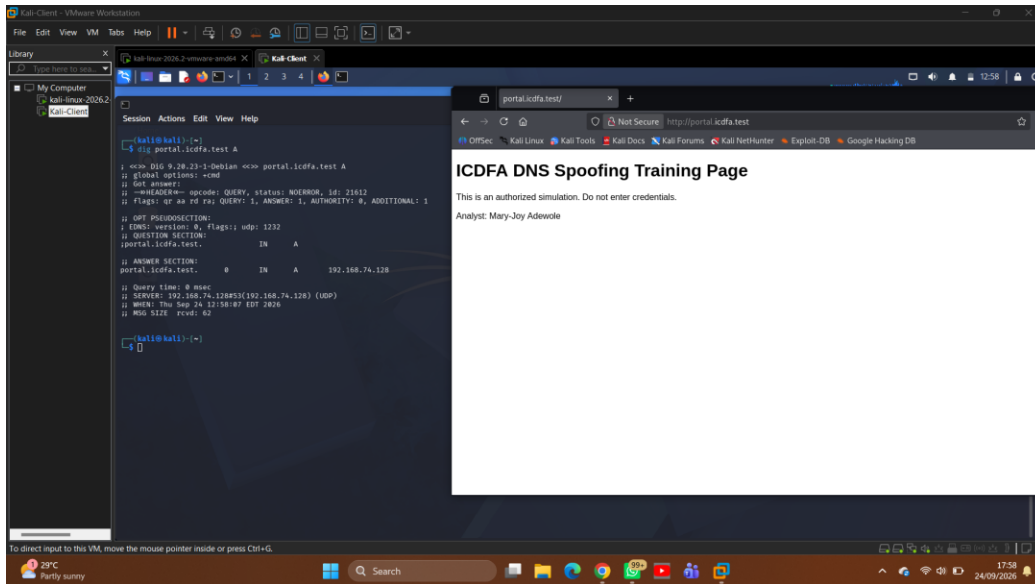



Figure 15: Victim's dig query resolved (NOERROR, answer 192.168.74.128) and the training page loaded in-browser during the poisoning window.

5.1 Packet-level evidence

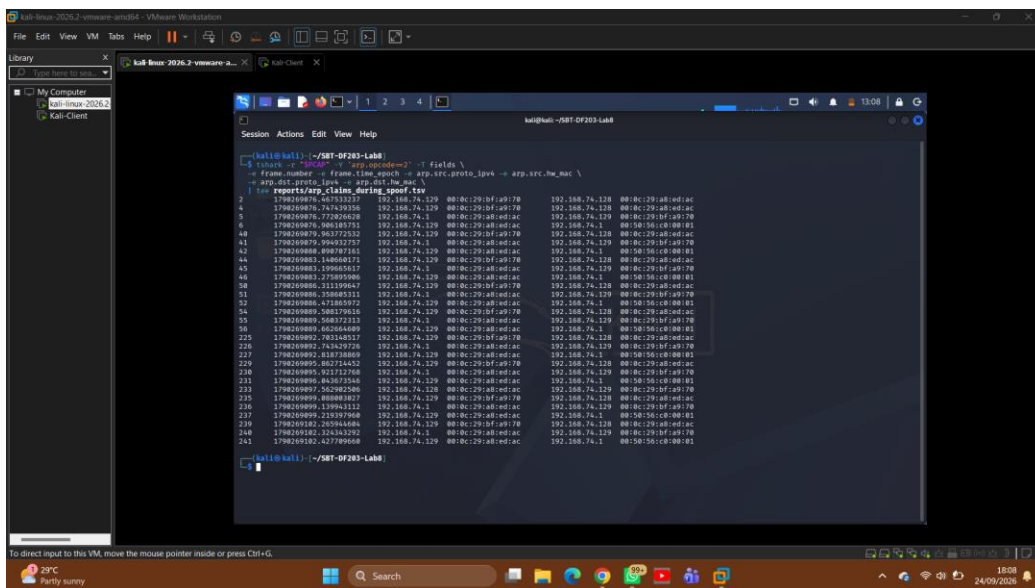


Figure 16: ARP opcode-2 (reply) packets captured during the event, showing the analyst's MAC (00:0c:29:a8:ed:ac) falsely claimed as both the gateway (192.168.74.1) and the victim (192.168.74.129) — the two-way ARP poisoning signature.

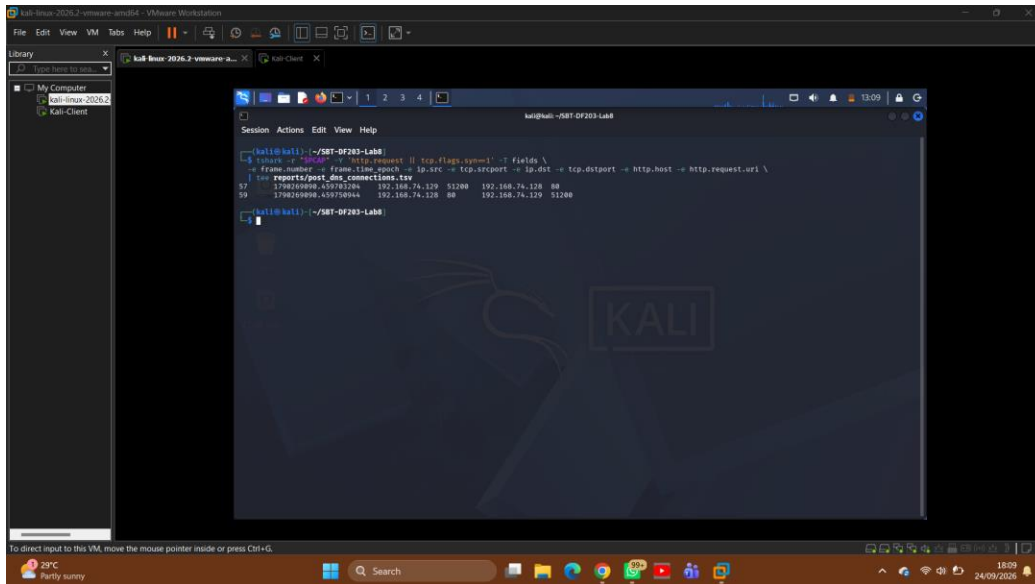


Figure 17: Victim's subsequent TCP/HTTP connection to the analyst web server (192.168.74.129:51200 → 192.168.74.128:80), immediately following DNS resolution.

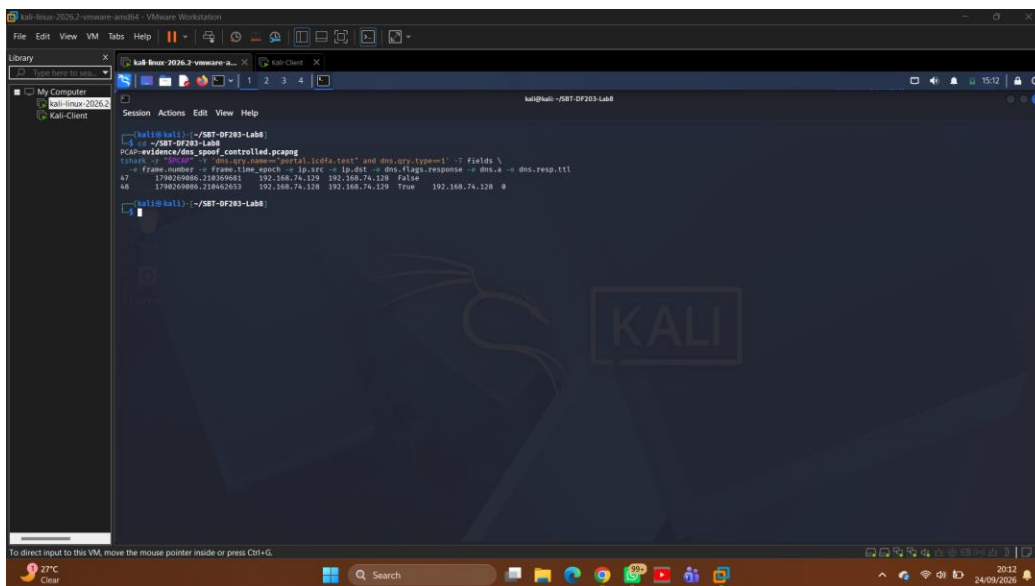


Figure 18: The victim's A-record query/response pair for portal.icdfa.test (frames 47–48), timestamped inside the active poisoning window, answer 192.168.74.128, TTL 0.

6. ARP Correlation

The captured ARP replies (Figure 16) confirm that during the event, the gateway IP (192.168.74.1) and the victim IP (192.168.74.129) were both falsely mapped, in the opposing party's ARP table, to the analyst's own MAC address (00:0c:29:a8:ed:ac). This is the standard man-in-the-middle positioning mechanism that DNS spoofing over a LAN depends on: with both parties' traffic routed through the analyst, in-flight DNS responses become interceptable.

7. Victim Connection Correlation

Figure 17 shows the victim initiating a TCP connection to 192.168.74.128:80 (the analyst's web server) immediately after resolving portal.icdfa.test, and the victim's browser (Figure 15) subsequently rendered the ICDFA training warning page. This demonstrates the full causal chain: ARP poisoning established the MITM position, DNS resolution completed while that position was active, and the victim's follow-on HTTP request reached the analyst rather than any independent host.

8. Alternative Explanations and Confidence Level

A significant, and honestly-reported, limitation of this run: because scripts/dns_spoof.py's corrected hostDict maps portal.icdfa.test to the same IP address (192.168.74.128) that the legitimate training resolver (dnsmasq) already returns, the DNS answer itself is identical whether or not interception occurred. The forensic signal in this run is therefore not a divergent DNS answer, but the ARP-layer evidence of MITM positioning (Section 6) combined with the timing correlation between poisoning, resolution, and the HTTP connection (Section 7).

Alternative explanation considered: normal DNS caching or a legitimate second resolver could in principle produce a duplicate response for one query. This is ruled out here because the duplicate/competing-response scenario described in the lab brief was not observed for the A record (only a single, correctly matched query/response pair was found for dns.qry.type==1); the actual anomaly is at the ARP layer, not the DNS layer, in this run.

Confidence level: High, for ARP poisoning having occurred and having established a MITM position between the victim and the analyst, and for the victim's subsequent connection reaching the analyst as a direct result. Confidence is not applicable for "forged vs legitimate DNS answer" in the narrow sense, since the two were configured to coincide in this run.

8.1 Baseline vs. Controlled Spoof Comparison

Indicator	Baseline	Controlled Spoof	Assessment
DNS responder IP	192.168.74.128	192.168.74.128	Same IP; see Section 8 note
Transaction ID match	Matched (baseline capture)	Matched (0x8e98)	Normal in both
Answer IP	192.168.74.128	192.168.74.128	Identical (by design, see Section 8)
TTL	0	0	Same
Competing answers	No	No (for A record)	No DNS-layer race observed
Gateway ARP mapping	Legitimate	Forged — analyst MAC (00:0c:29:a8:ed:ac)	Confirmed poisoning
Victim ARP mapping (at gateway)	Legitimate	Forged — analyst MAC (00:0c:29:a8:ed:ac)	Confirmed poisoning
Subsequent HTTP destination	N/A	192.168.74.128:80 (analyst)	Victim reached analyst server

9. Cleanup and Defensive Recommendations

9.1 Cleanup and verification

The lab environment underwent an unplanned restart of the analyst VM partway through the exercise, after the controlled capture had already been completed and hashed. Post-restart state was checked and found consistent with a clean state (IP forwarding disabled, no leftover iptables rules, no spoofing processes running); this was confirmed with a further explicit cleanup pass.

- Correlate DNS, ARP, endpoint, switch, proxy, and TLS logs during incident investigation rather than relying on any single data source.

10. Appendix

Appendix A — Corrected dns_spoof.py hostDict

```
hostDict = {  
    b"portal.icdfa.test.": "192.168.74.128"  
}
```

Appendix B — Command reference

Command / Field	Purpose
dns.flags.response	Distinguishes DNS query vs. response
dns.id	Matches query to response (transaction ID)
dns.a	IPv4 address returned in an A record answer
dns.resp.ttl	TTL of the DNS answer
arp.src.proto_ipv4 / hw_mac	IP-to-MAC claim made by an ARP reply
http.host	Requested HTTP Host header

Appendix C — Student Declaration

I confirm that I completed this lab in an authorized, host-only environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions. — Mary-Joy Adewole, 2025/FWSD/11468.